ŽILINSKÁ UNIVERZITA V ŽILINE

FAKULTA RIADENIA A INFORMATIKY

BAKALÁRSKA PRÁCA

JÁN GAJDOŠÍK

Softvérový nástroj na zber a vyhodnocovanie prác študentov

Vedúci práce: Ing. Peter Sedláček

Registračné číslo: 1414/2020

Žilina, 2021

ŽILINSKÁ UNIVERZITA V ŽILINE

FAKULTA RIADENIA A INFORMATIKY

BAKALÁRSKA PRÁCA

ŠTUDIJNÝ ODBOR: INFORMATIKA

JÁN GAJDOŠÍK

Softvérový nástroj na zber a vyhodnocovanie prác študentov

Žilinská univerzita v Žiline

Fakulta riadenia a informatiky

Katedra informatiky

Žilina, 2021

ŽILINSKÁ UNIVERZITA V ŽILINE, FAKULTA RIADENIA A INFORMATIKY.

ZADANIE TÉMY BAKALÁRSKEJ PRÁCE.

Študijný program: Informatika

Meno a priezvisko
Ján Gajdošík

Osobné číslo
557965

Názov práce v slovenskom aj anglickom jazyku

Softvérový nástroj na zber a vyhodnocovanie prác študentov

Software tool for collection and evaluation of students works

Zadanie úlohy, ciele, pokyny pre vypracovanie (Ak je málo miesta, použite opačnú stranu)

Cieľ bakalárskej práce:
Cieľom práce je implementovať nástroj na zber a vyhodnocovanie prác študentov.

Obsah:
V rámci práce je potrebné upraviť existujúci systém na zber a vyhodnocovanie študentských prác. Je potrebné navrhnúť a implementovať webovú aplikáciu, ktorá umožní:

- Nahrávať a spravovať študentské práce. Systém by mal podporovať nahrávanie viacerých súborov od jedného študenta.
- Hodnotiť práce. Je potrebné implementovať funkcionalitu, ktorá umožní učiteľovi hodnotiť tieto práce, pridávať komentáre a poznámky.
- Kontrolovať originalitu prác. Na kontrolu originality jednotlivých prác je vhodné vybrať a použiť existujúci algoritmus.

Meno a pracovisko vedúceho BP: Ing. Peter Sedláček, KI, ŽU
Meno a pracovisko tutora BP:

08 FEB. 2021

vedúci katedry
(dátum a podpis)

Zadanie zaregistrované dňa 29.10.2020 pod číslom 1414/2020 podpis

Poďakovanie

Moje poďakovanie patrí školiteľovi Ing. Petrovi Sedláčkovi, za odbornú pomoc, pripomienky a usmerňovanie pri tvorbe práce.

ABSTRAKT V ŠTÁTNOM JAZYKU

GAJDOŠÍK, JÁN: *Softvérový nástroj na zber a vyhodnocovanie prác študentov*. [Bakalárska práca]. – Žilinská univerzita v Žiline. Fakulta riadenia a informatiky; Katedra informatiky. – Školiteľ/Vedúci: Ing. Peter Sedláček – Stupeň odbornej kvalifikácie: bakalár. – Žilina: FRI ŽU v Žiline, 2021. Počet strán - 45

Hlavným cieľom našej bakalárskej práce bolo vytvoriť systém na zber a vyhodnocovanie prác študentov. V rámci práce je opísaný postup tvorby od stanovenia požiadaviek na systém cez návrh a výber technológií až po samotnú implementáciu. Navrhovaný systém sme sa rozhodli implementovať ako webovú aplikáciu s osobitným rozhraním pre učiteľov a študentov. Vyučujúci sú schopní vytvárať, modifikovať a prideľovať študentom zadania a následne študentské práce hodnotiť. Študenti budú prostredníctvom aplikácie odovzdávať vypracované zadania a kontrolovať svoje výsledky. Systém disponuje aj možnosťou kontroly originality pri niektorých typoch prác. Samotná webová aplikácia pracuje na princípe komunikácie medzi stranou klienta a servera. Klient posiela požiadavky na server, z ktorého sú v odpovediach odosielané dáta získané z databázy. Tie klient následne využíva pri tvorbe výsledného dokumentu, ktorý je zobrazený užívateľovi. Pri implementácií sa opierame o rôzne technológie ako PostgreSQL a frameworky Laravel s Reactom.

Kľúčové slová: systém, webová aplikácia, webová stránka, klient, server, databáza, JavaScript, JSX, PHP

ABSTRAKT V CUDZOM JAZYKU

The main goal of our bachelor thesis was to create system for collecting and evaluating students' assignments. The thesis desciribes the process of creation from setting system requirements through the design of application and selection of technologies up to the implementation itself. We decided to implement the proposed system as a web application with a distinctive interface for teachers and students. Teachers are able to create, modify and assign tasks to students and then evaluate students' assignments. Students can submit the completed assignments and check their results through the application. The system also implements the functionality for the evaluation of originality for several types of assignments. The web application works on the principle of communication between the client and the server. The client sends requests to the server, which sends back responses with data obtained from the database. These are then used to create resluting document, which is displayed to the user on the client side. In implementation, we rely on various technologies such as PostgreSQL and Laravel + React frameworks.

Key words: system, web application, web page, client, server, database, JavaScript, JSX, PHP

Obsah

Obsah **7**
Zoznam obrázkov **9**
Zoznam skratiek **10**
Úvod **11**
1 Ciele práce **12**
1.1 Požiadavky 12
2 Modely prípadov použitia **13**
2.1 Správa zadaní 13
2.1.1 Scenár – Vytvorenie zadania 15
2.1.2 Scenár – Pridelenie zadania 15
2.1.3 Scenár – Hodnotenie prác 16
2.2 Odovzdávanie prác 17
2.2.1 Scenár – odovzdanie zadania 18
2.2.2 Scenár – náhľad hodnotenia 18
3 Návrh **19**
3.1 Analýza 19
3.2 Požiadavky 19
3.2.1 Databáza 19
3.2.2 Server 19
3.2.3 Klient 20
3.3 Databáza – dátový model 21
3.3.1 Tabuľky 22
3.4 Architektúra systému 25
4 Použité technológie **26**
4.1 DbDiagram 26
4.2 PostgreSQL 26
4.3 Laravel 26
4.3.1 MVC Architektúra 26
4.3.2 Routing 27
4.3.3 Composer 27
4.3.4 Artisan 27
4.4 JWT 27
4.5 Bootstrap 28
4.6 React 28
4.6.1 NPM 28

4.7 JPlag 29
5 Implementácia 30
5.1 Databáza 30
5.2 Úvod do projektu 30
5.3 Autentifikácia 31
5.3.1 Server 31
5.3.2 Klient 32
5.3.3 Proces prihlásenia a odhlásenia 33
5.4 Učiteľské rozhranie 35
5.4.1 Zadania 35
5.4.2 Kontrola originality prác a hodnotenie 37
5.5 Študentské rozhranie 38
5.5.1 Odovzdávanie prác a výsledky 39
Záver 40
Zoznam použitej literatúry 41
Zoznam príloh 43
Prílohy 44
Príloha A: Používateľská príručka 45
Príloha B: DVD 46

Zoznam obrázkov

Obrázok 1: Správa zadaní 13
Obrázok 2: Odovzdávanie prác 17
Obrázok 3: Požiadavky servera 20
Obrázok 4: Požiadavky z pohľadu učiteľa 20
Obrázok 5: Požiadavky z pohľadu študenta 21
Obrázok 6: Spoločné užívateľské požiadavky 21
Obrázok 7: Pôvodný databázový model 22
Obrázok 8: Aktuálny databázový model 24
Obrázok 9: REST API - LARAVEL architektúra [2] 25
Obrázok 10: Trieda ApiHelper 33
Obrázok 11: UserController - funkcia login() 34
Obrázok 12: Prihlasovanie 34
Obrázok 13: Požiadavka na server o vytvorenie nového zadania 35
Obrázok 14: Prideľovanie zadania študentom na strane klienta 36
Obrázok 15: Spracovanie požiadavky na pridelenie zadania študentom na serveri 37
Obrázok 16: Náhľad hodnotenia práce študenta 39

Zoznam skratiek

API	Application Programming Interface
CRUD	Create, read, update and delete
GUI	Graphical User Interface
JWT	JSON Web Token
HTTP	Hypertext Transfer Protocol
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JSX	JavaScript + XML
MVC	Model View Controller
REST	Representational State Transfer
ORM	Object–relational mapping
NPM	Node Package Manager

Úvod

V súčasnosti je elektronická forma odovzdávania, uchovávania a spracovávania dokumentov nie len trendom, ale aj nevyhnutnou súčasťou viacerých inštitúcií. Jednou z nich je aj Fakulta riadenia a informatiky v Žiline. Nakoľko sa jedná a školu so zameraním na informatiku, veľa materiálov je zo svojej podstaty vhodnejšie uchovávať v elektronickej podobe. Na základe tejto skutočnosti sú vyžadované napríklad softvérové nástroje pre správu prác študentov. Momentálne existuje niekoľko nástrojov pre zber a uchovávanie prác. Naša fakulta využíva vzdelávaciu open-source platformu Moodle [1], ktorá poskytuje systém na vytváranie personalizovaných učebných prostredí. Nakoľko každý predmet má však svoje špecifiká a požiadavky, univerzálny softvér nemusí byť vyhovujúcim pre celú fakultu.

Z tohto dôvodu sa stalo cieľom našej bakalárskej práce vytvorenie webovej aplikácie, ktorá bude tieto požiadavky zastrešovať. Základnou funkciou bude nahrávanie prác zo strany študentov a ich hodnotenie zo strany vyučujúcich. Ďalšou funkcionalitou aplikácie bude možnosť vytvárania nových zadaní, ako aj ich úprava a mazanie, prípadne nastavenie viditeľnosti, v prípade, že už zadanie nebude aktuálne. Nadväzujúca funkčnosť bude v prideľovaní týchto zadaní študentom. Študent bude schopný si zadanie otvoriť, prípadne stiahnuť a následne svoju prácu odovzdať. Po ohodnotení vyučujúcim si bude môcť pozrieť svoj výsledok spolu s komentárom. Práce budú naviac kontrolované z hľadiska originality.

Naša bakalárska práca bude rozdelená na niekoľko častí. V prvej časti sa budeme zaoberať cieľmi a požiadavkami na systém. Pokračovať budeme modelmi prípadov použitia, kde sa zameriame na hlavné scenáre. Ďalšia kapitola bude zložená z troch častí, analýza, požiadavky a samotný návrh jednotlivých celkov. V nasledujúcej sekcií si priblížime použité technológie. Nadväzujúca kapitola bude venovaná samotnej implementácií a ukážkam GUI. Záver práce bude obsahovať súhrnný popis riešenia.

1 Ciele práce

V úvodnej časti práce sa budeme zaoberať hlavými cieľmi a požiadavkami, ktoré boli stanovené v samom začiatku ako podklad pre následnú tvorbu systému pre fakultný predmet Princípy operačných systémov.

V rámci štúdia tohto kurzu majú študenti naplánované dva semestrálne zápočty. Výsledkom prvého, sú skripty v jazyku shell. Druhý je zameraný na programátorské schopnosti a znalosti v jazyku C / C++. Ďalšou povinnou súčasťou predmetu je úspešné spracovanie a obhajoba semestrálnej práce rovnako v jazyku C. Dobrovoľnou aktivitou sú domáce úlohy, riešené na základe zadania. Výsledkom všetkých častí môže byť teda jeden alebo viacero súborov so zdrojovými kódmi, v prípade semestrálnej práce navyše dokumentácia. Z tohto dôvodu je naším hlavným cieľom umožniť zhromažďovanie, spracovanie a vyhodnocovanie týchto študentských prác.

1.1 Požiadavky

Pri stanovovaní cieľov a požiadaviek sme vychádzali priamo zo zadania našej práce, kde boli spomenuté niektoré hlavné funkcionality. Okrem týchto cieľov nám vznikli aj ďalšie. Všetky základné požiadavky na našu aplikáciu sa dajú zhrnúť v nasledujúcich bodoch:

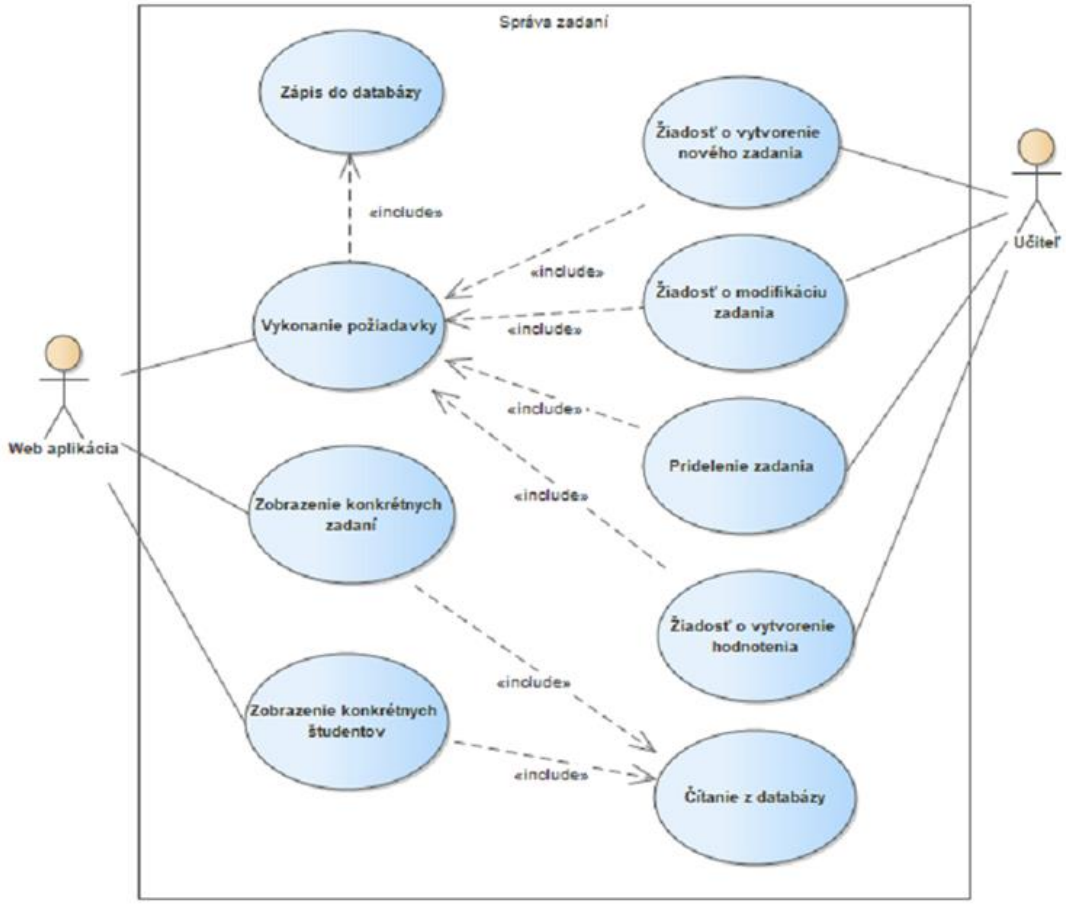
- Vytvárať a modifikovať zadania.
- Prideľovať zadania jednotlivým študentom.
- Hodnotiť práce a pridávať komentáre.
- Nahrávať a spravovať študentské práce.
- Kontrolovať originalitu prác. Na kontrolu originality jednotlivých prác je vhodné vybrať a použiť existujúci algoritmus.

2 Modely prípadov použitia

V tejto časti si priblížime modely prípadov použitia spolu s hlavnými scenármi, ktoré popisujú realizáciu jednotlivých prípadov použitia v nami navrhovanom riešení.

2.1 Správa zadaní

Jednou z hlavných funkcií aplikácie bude správa zadaní, základné CRUD operácie a zobrazovanie zadaní podľa konkrétnych kritérií. Diagram so všetkými prípadmi použitia je možné vidieť na obrázku 5. Tento diagram obsahuje nasledujúce elementy:



Obrázok 1: Správa zadaní

Učiteľ – v tomto prípade len používateľ so statusom učiteľ, bude schopný vykonať nasledujúce akcie.

Vytvorenie nového zadania – užívateľ vyplní formulár, kde bude mať možnosť výberu typu zadania (prvý zápočet, druhý zápočet, domáca úloha alebo semestrálna práca),

vyplní položku názov, pridá popis, zvolí termín odovzdania a v prípade potreby priloží súbor s konkrétnym znením zadania. Vyplnený formulár následne uloží.

Žiadosť o modifikáciu zadania – užívateľ bude mať možnosť svoje zadanie modifikovať, ak bude mať záujem o ľubovoľnú zmenu, rovnako ak ešte zadanie nepridelil študentom, alebo pridelil a pridelenie študentov zrušil, bude môcť takéto zadanie vymazať.

Pridelenie zadania – užívateľ svoje zadania bude môcť prideľovať študentom svojich študijných skupín.

Žiadosť o vytvorenie hodnotenia – pokiaľ študent odovzdal svoju prácu na hodnotenie, užívateľ (učiteľ) bude mať možnosť prácu lokálne stiahnuť, vyplniť formulár s počtom bodov a pridať komentár. Taktiež bude môcť pre niektoré typy zadaní spustiť kontrolu originality odovzdaných prác.

Vykonanie požiadavky – spočíva v tom, že aplikácia pošle cez API rozhranie požiadavku na server pomocou konkrétnej metódy spolu s príslušnými dátami, keď server príjme požiadavku, autentifikuje užívateľa a skontroluje validitu dát, v prípade úspechu ak sa v požiadavke nachádzajú dáta na zápis odošle ich na uloženie do databázy, odpoveď je následne odoslaná na stranu klienta.

Zápis do databázy – dáta zo servera sú uložené do konkrétneho objektu (tabuľky) databázy.

Zobrazenie konkrétnych zadaní – užívateľovi s kompetenciami učiteľa, budú zobrazené jeho zadania s možnosťou ich modifikácie alebo zadania všetkých vyučujúcich bez možnosti modifikácie.

Zobrazenie konkrétnych študentov – ku všetkým zadaniam, ktoré bude mať používateľ k dispozícii, bude viditeľný aj zoznam študentov, ktorí sú k danému zadaniu pridelení spolu s informáciou o ich aktuálnom stave odovzdávania

Čítanie z databázy – dáta budú na základe užívateľských požiadaviek načítavané z databázy.

2.1.1 Scenár – Vytvorenie zadania

1. Užívateľ sa prihlási do svojho konta na webovej stránke.
2. Systém užívateľa presmeruje na úvodnú stránku predmetu.
3. V menu užívateľ klikne na položku „Moje zadania“.
4. Systém zobrazí všetky zadania vytvorené prihláseným používateľom.
5. Užívateľ klikne na tlačidlo „Nové zadanie +“.
6. Systém zobrazí formulár na vyplnenie pre nové zadanie.
7. Užívateľ vyplní všetky povinné položky formulára .
8. Vyplnený formulár užívateľ odošle kliknutím na tlačidlo „Ulož“.
9. Systém vykoná požiadavku.
10. Systém presmeruje užívateľa na stránku jeho zadaní a zobrazí notifikačné okno o úspešnom vytvorení.

2.1.2 Scenár – Pridelenie zadania

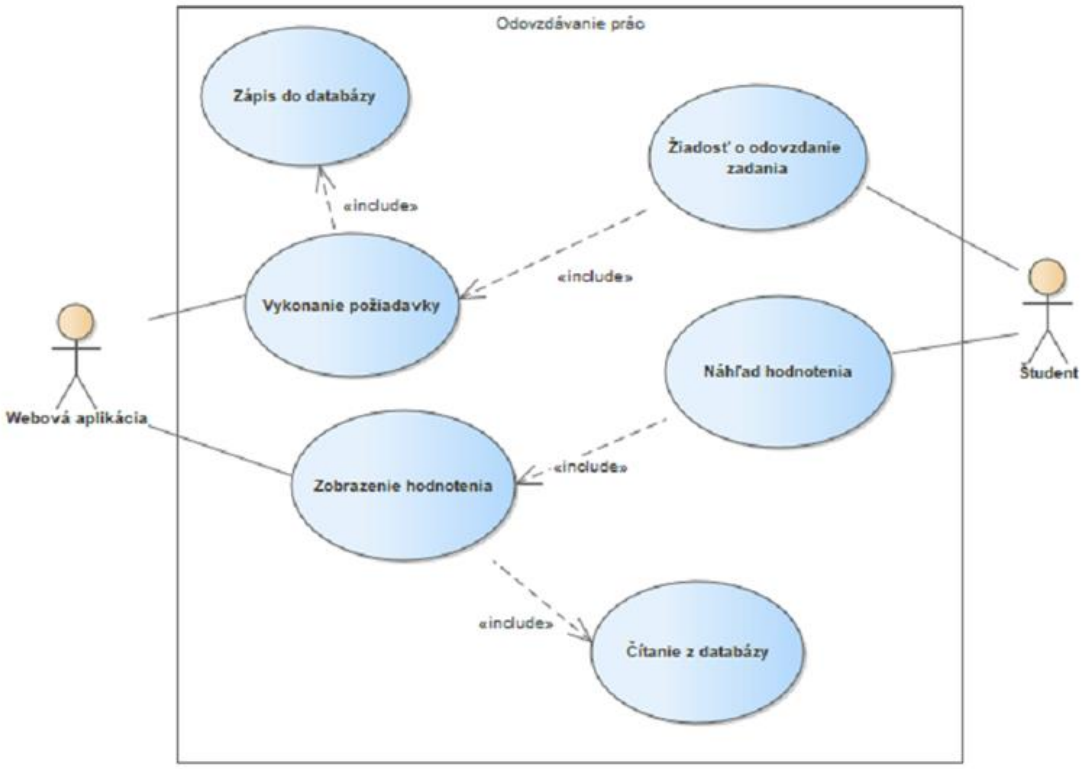
1. Užívateľ sa prihlási do svojho konta na webovej stránke.
2. Systém užívateľa presmeruje na úvodnú stránku predmetu.
3. V menu užívateľ klikne na položku „Moje zadania“.
4. Systém zobrazí všetky zadania vytvorené prihláseným používateľom.
5. Užívateľ si vyberie konkrétne zadanie zo zoznamu, ktoré chce priradiť a klikne na ikonu znázorňujúcu pridelenie študentov.
6. Systém zobrazí všetky cvičenia daného užívateľa a študentov, ktorým už je zadanie pridelené.
7. Užívateľ si zobrazí študentov, ktorých chce prideliť stlačením tlačidla „+“ pri konkrétnom cvičení/cvičeniach.
8. Systém zobrazí doteraz nepridelených študentov daného cvičenia.
9. Ak užívateľ nechce prideliť všetkých študentov daného cvičenia, kliknutím na tlačidlo „-“ pri konkrétnom študentovi ho odoberie z aktuálneho priradenia.
10. Užívateľ pridelí zadanie všetkým zobrazeným študentom, kliknutím na tlačidlo „Ulož“
11. Systém vykoná požiadavku.
12. Systém presmeruje užívateľa na stránku pridelených študentov k danému zadaniu.

2.1.3 Scenár – Hodnotenie prác

1. Užívateľ sa prihlási do svojho konta na webovej stránke.
2. Systém užívateľa presmeruje na úvodnú stránku predmetu.
   a. V menu užívateľ klikne na položku „Moje zadania“.
      3. Systém zobrazí všetky zadania vytvorené prihláseným používateľom.
   b. V menu užívateľ klikne na položku „Všetky zadania“.
      3. Systém zobrazí zadania všetkých vyučujúcich predmetu.
4. Užívateľ si vyberie konkrétne zadanie zo zoznamu, v ktorom chce hodnotiť práce študentov a klikne na ikonu znázorňujúcu hodnotenie.
5. Systém zobrazí tabuľku všetkých študentov pridelených k vybranému zadaniu spolu s tlačidlom pre kontrolu originality prác k danému zadaniu, pokiaľ zadanie takouto kontrolou doteraz neprešlo.
6. Systém zobrazí rovnako tlačidlá „Hodnotiť“ pri študentoch, ktorí už prácu odovzdali.
7. Užívateľ môže odovzdané práce odoslať na kontrolu originality kliknutím na tlačidlo „Kontrola zhody“.
8. Užívateľ si ďalej z množiny vypísaných študentov vyberie jedného a klikne na tlačidlo „Hodnotiť“.
9. Systém zobrazí informácie o zadaní a študentovi spolu s položkou na stiahnutie súboru a formulárom pre hodnotenie.
10. Užívateľ po kliknutí na tlačidlo „Otvor“ stiahne lokálne odovzdanú prácu do svojho počítača.
11. Užívateľ vyplní hodnotiaci formulár.
12. Vyplnený formulár užívateľ odošle kliknutím na tlačidlo „Ulož“.
13. Systém vykoná požiadavku.
14. Systém presmeruje užívateľa späť na tabuľku pridelených študentov k danému zadaniu.

2.2 Odovzdávanie prác

Druhou hlavnou funkciou aplikácie bude študentská časť slúžiaca na odovzdávanie prác a zobrazenie svojich priebežných výsledkov. Diagram s jednotlivými prípadmi použitia je možné vidieť na obrázku 6. Tento diagram obsahuje nasledujúce elementy:



Obrázok 2: Odovzdávanie prác

Študent – v tomto prípade len používateľ so statusom študent, bude schopný vykonať nasledujúce akcie.

Žiadosť o odovzdanie zadania – užívateľ vyplní formulár, pridá IP adresu užívateľovho počítača v sieti, z ktorého je daná práca odosielaná a priloží vypracovanú prácu v požadovanom formáte , vyplnený formulár uloží.

Náhľad hodnotenia – študent si bude môcť svoj výsledok pri konkrétnom zadaní pozrieť, akonáhle bude práca vyučujúcim ohodnotená.

Vykonanie požiadavky – spočíva v tom, že aplikácia pošle cez API rozhranie požiadavku na server pomocou konkrétnej metódy spolu s príslušnými dátami, keď server príjme požiadavku, autentifikuje užívateľa a skontroluje validitu dát, v prípade úspechu ak sa v požiadavke nachádzajú dáta na zápis odošle ich na uloženie do databázy, odpoveď je následne odoslaná na stranu klienta.

Zápis do databázy – dáta zo servera sú uložené do konkrétneho objektu (tabuľky) databázy.

Zobrazenie hodnotenia – študentovi s kompetenciami študenta, bude k dispozícií možnosť náhľadu hodnotenia jeho práce spolu s výsledným počtom bodov a komentárom.

Čítanie z databázy – dáta budú na základe užívateľských požiadaviek načítavané z databázy.

2.2.1 Scenár – odovzdanie zadania

1. Užívateľ sa prihlási do svojho konta na webovej stránke.
2. Systém užívateľa presmeruje na úvodnú stránku predmetu.
3. V menu užívateľ klikne na položku „Pridelené zadania“.
4. Systém zobrazí tabuľku zadaní, ktoré sú užívateľovi pridelené.
5. Užívateľ si vyberie konkrétne zadanie, ku ktorému chce svoju prácu odovzdať a klikne na položku „Odovzdaj“.
6. Systém zobrazí formulár pre vyplnenie IP adresy a nahratie súboru.
7. Užívateľ vyplní údaje a nahrá súbor.
8. Užívateľ odovzdá prácu stlačením tlačidla „Ulož“.
9. Systém vykoná požiadavku.
10. Systém presmeruje užívateľa späť na tabuľku pridelených zadaní.

2.2.2 Scenár – náhľad hodnotenia

1. Užívateľ sa prihlási do svojho konta na webovej stránke.
2. Systém užívateľa presmeruje na úvodnú stránku predmetu.
3. V menu užívateľ klikne na položku „Pridelené zadania“.
4. Systém zobrazí tabuľku zadaní, ktoré sú užívateľovi pridelené.
5. Systém zobrazí rovnako tlačidlá „Výsledok“ pri vypracovaných zadaniach, ktoré už boli hodnotené.
6. Užívateľ klikne na položku „Výsledok“.
7. Systém zobrazí notifikačné okno s počtom dosiahnutých bodov študenta a komentárom k práci od vyučujúceho.

3 Návrh

Túto kapitolu bakalárskej práce si rozdelíme na tri časti. V úvodnej sa budeme zaoberať analýzou jednotlivých celkov, na ktoré sme si systém rozčlenili. Pokračovať budeme požiadavkami na tieto komponenty. V záverečnej tretej časti sa budeme venovať samotnému návrhu databázy a architektúre systému spolu s návrhom klientskeho a serverového rozhrania.

3.1 Analýza

Nástrojom na zber a vyhodnocovanie prác študentov bude webová aplikácia. Konštrukciu samotnej aplikácie si rozdelíme na tri časti:

- Databáza - dátový model uchovávajúci všetky potrebné dáta.
- Server – back-end aplikácie umožňujúci prácu s databázou a celkovou logikou fungovania aplikácie.
- Klient – front-end aplikácie umožňujúci interakciu medzi užívateľom a dátami získanými zo servera prostredníctvom GUI. Používateľské rozhranie je rôzne v závislosti od role používateľa aplikácie, t.j. rozdielne pre učiteľov a pre študentov.

3.2 Požiadavky

3.2.1 Databáza

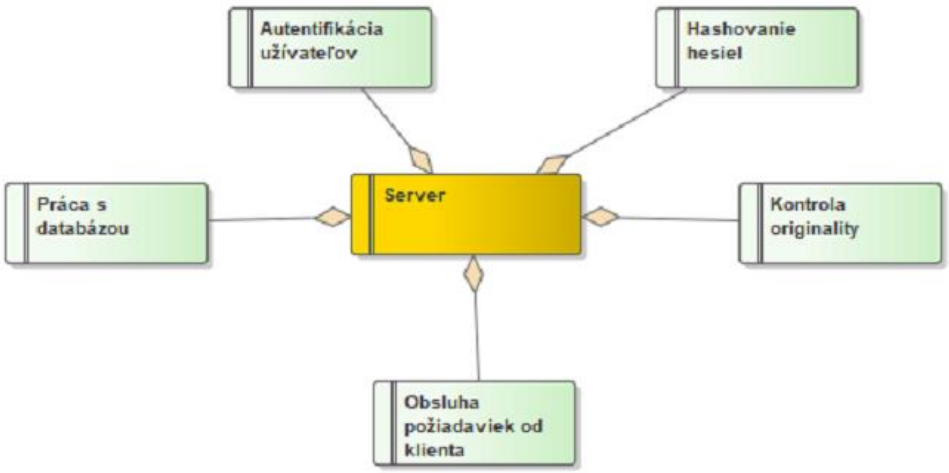
Základným stavebným komponentom celého systému je databáza, ktorá bude uchovávať veľké množstvo dát. V procese výstavby systému je na prvom mieste práve jej logická schéma. Správne navrhnutá schéma relačných modelov (tabuliek) spolu s relačnými vzťahmi, je to od čoho sa bude systém odvíjať počas celého svojho fungovania.

Hlavnou požiadavkou bude vykonávanie základných CRUD operácií a získavanie požadovaných dát nad dátovým modelom. Tieto funkcie sú nevyhnutné pre funkčnosť aplikácie.

3.2.2 Server

Back-end je serverová časť aplikácie, ktorú bežný používateľ nevidí, je pred ním skrytá a nemôže do nej zasahovať. Spolupracuje zároveň s databázou.

Medzi naše požiadavky na back-end časť aplikácie môžeme zaradiť prácu s databázou, obsluha požiadaviek od klienta, autentifikáciu užívateľov, hashovanie hesiel, kontrolu originality prác a v podstate celú logiku fungovania aplikácie.

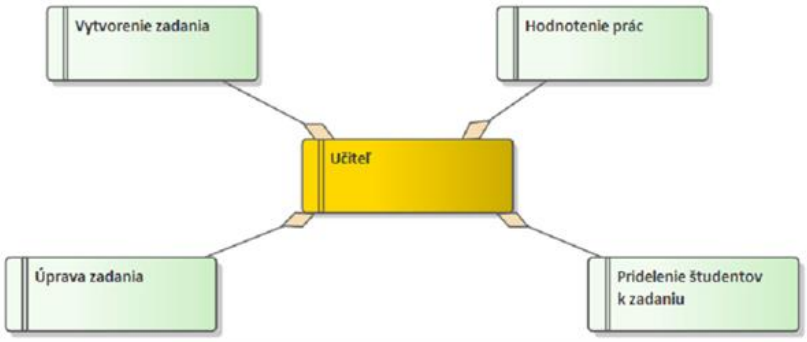


Obrázok 3: Požiadavky servera

3.2.3 Klient

Front-end je klientska časť aplikácie, s ktorou je užívateľ priamo v kontakte. Môžeme ju nazývať aj GUI, slúži na komfortnejší prístup a uľahčenie práce so softvérom. V našej práci budeme rozhranie deliť na základe kompetencií na rozhranie učiteľov a študentov.

Hlavnými požiadavkami učiteľského rozhrania by mala byť možnosť vytvárať nové zadania, spolu s modifikáciou a možnosťou ich prideľovania ku konkrétnym študentom. Odovzdané práce budú môcť následne otvoriť a hodnotiť.



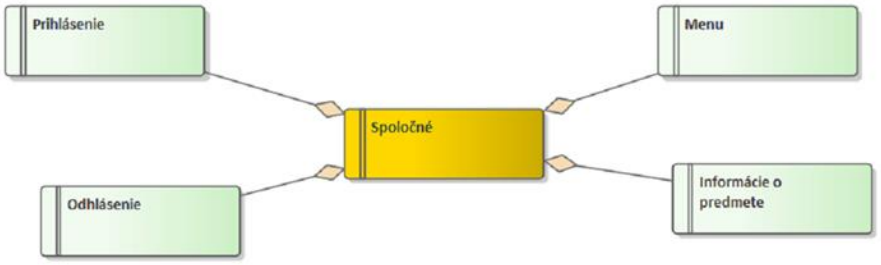
Obrázok 4: Požiadavky z pohľadu učiteľa

Medzi požiadavky pre študentské rozhranie zaraďujeme možnosť náhľadu ku pridelenému zadaniu a priestor pre odovzdanie svojej práce.



Obrázok 5: Požiadavky z pohľadu študenta

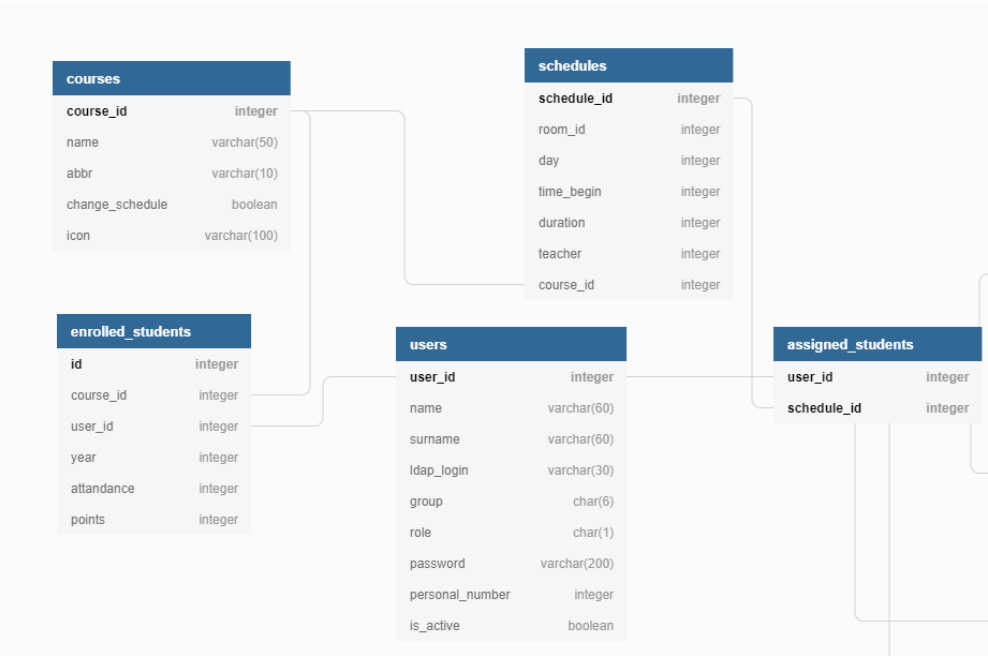
Medzi spoločné požiadavky môžeme zaradiť funkcionalitu, ktorá zabezpečí aby sa užívatelia dokázali prihlásiť pomocou svojich prihlasovacích údajov a dodatočne odhlásiť. Aplikácia by mala obsahovať kvôli lepšej orientácií komponent menu, kde jednou z položiek budú základné informácie o predmete.



Obrázok 6: Spoločné užívateľské požiadavky

3.3 Databáza – dátový model

V nasledujúcej podkapitole sa budeme venovať návrhu entitno-relačného modelu. Databáza bude vychádzať z už existujúceho databázového modelu, ktorý je využívaný na fakulte riadenia a informatiky pre účely prihlasovania sa na cvičenia. Pri návrhu databázy si ako prvé stanovíme účel databázy, a teda, aby uchovávala všetky potrebné dáta pre fungovanie systému. Zahŕňané budú informácie o všetkých používateľoch, z ktorých dáta budú potrebné napríklad pre prihlásenie užívateľov do systému, zapísaných študentoch, ktorí budú vychádzať práve z tabuľky všetkých užívateľov, predmetoch, rozvrhoch, zadaniach, ktoré budú vytvárané a prideľované študentom, odovzdaných zadaniach od študentov, ďalej výsledkoch, a teda hodnoteniach prác, originalitách a vzájomných vzťahoch medzi entitami. V práci budeme využívať voľne dostupný databázový systém PostgreSQL.



Obrázok 7: Pôvodný databázový model

3.3.1 Tabuľky

users – tabuľka bude uchovávať všetkých užívateľov, učiteľov aj študentov a pre každého bude sekvenčne generovaný primárny kľúč. Z tabuľky vychádzajú aj prihlasovacie údaje, ktorými budú prihlasovacie meno a heslo. Po úspešnej autentifikácií bude vyplnený údaj o tokene. Dôležitým údajom bude rola užívateľa, a teda, či je daný užívateľ učiteľ alebo študent. Ďalšími atribútmi budú meno, priezvisko, osobné číslo, štatút či je užívateľ aktuálne prihlásený a študijná skupina v prípade študenta.

enrolled_students – ďalšou entitou dátového modelu budú zapísaní študenti pre daný akademický rok. Jeden záznam bude pozostávať zo sekvenčne generovaného primárneho kľúča, akademického roku, dochádzky a počtu dosiahnutých bodov. Nakoľko zapísaní študenti budú vychádzať z tabuľky všetkých užívateľov a budú im priradené konkrétne predmety, entita bude navyše obsahovať aj identifikátor užívateľa a identifikátor predmetu ako cudzí kľúč.

courses – tabuľka predmetov bude obsahovať záznamy o predmetoch vyučovaných na fakulte. Charakterizovaná bude svojim primárnym kľúčom, ktorým bude id predmetu. Záznamy budú obsahovať ďalej aj atribúty ako názov predmetu, skratku predmetu, atribút informujúci o zmene v rozvrhu a prípadný obrázok ako ikonu predmetu.

schedules – tabuľka bude uchovávať záznamy o cvičeniach konkrétneho predmetu v rozvrhu. Pre každé cvičenie bude sekvenčne generovaný primárny kľúč, cudzím kľúčom

v tejto tabuľke bude identifikátor predmetu, ku ktorému je cvičenie priradené. Ďalšími informáciami, ktoré bude tabuľka zahŕňať sú číslo miestnosti, v ktorej bude výučba prebiehať, deň v týždni, čas začiatku cvičenia a dĺžka trvania. Posledným atribútom bude vyučujúci, ktorému dané cvičenie prislúcha.

assigned_students – ku každému cvičeniu môže byť priradených viacero študentov a rovnako každému študentovi môže byť priradených viacero cvičení. Z toho dôvodu bude táto tabuľka reprezentovať asociatívny m:n vzťah medzi tabuľkou používateľov a tabuľkou cvičení. Obsahovať bude teda len unikátne identifikátory týchto dvoch tabuliek, ktoré budú tvoriť kompozitný primárny kľúč pre túto tabuľku.

tasks – entita bude uchovávať všetky zadania vytvorené vyučujúcimi. Nakoľko jedno zadanie môže mať len jedného autora, respektíve užívateľa, ktorý je tvorcom a užívateľ môže mať viacero vytvorených zadaní bude medzi tabuľkami užívateľov a zadaní vzťah 1:n. V tabuľke zadaní sa teda bude nachádzať identifikátor učiteľa. Medzi ďalšie informácie, ktoré bude tabuľka uchovávať môžeme zaradiť názov zadania, typ zadania, popis alebo znenie zadania, dátum vytvorenia a dátum dokedy treba zadanie vypracovať. V prípade, že vyučujúci bude chcieť nahrať súbor so zadaním, tabuľka bude navyše obsahovať aj cestu k súboru a jeho názov. Poslednými dvoma budú booleanovské atribúty, ktoré budú po vytvorení automaticky nastavené na hodnotu nepravda, jeden bude vyjadrovať, či užívateľ požiadal o skrytie zadania, druhý si bude uchovávať, či už pre konkrétne zadanie prebehla kontrola originality prác.

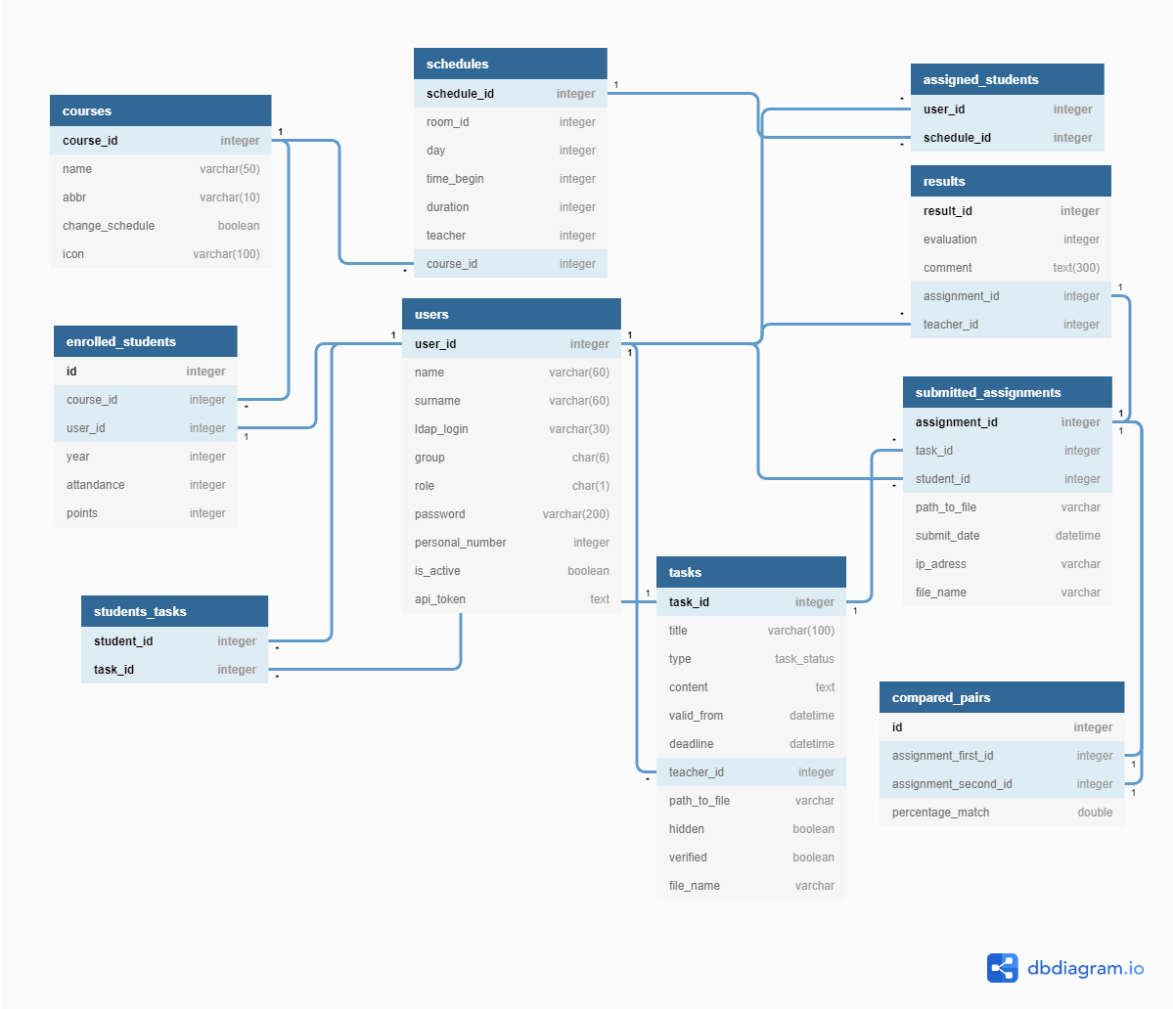
student_tasks – nasledujúca tabuľka bude obsahovať len dve položky, identifikátor študenta a identifikátor zadania, ktoré budú spolu vytvárať kompozitný primárny kľúč. Akonáhle vyučujúci priradí študenta ku zadaniu, vytvorí sa v tabuľke záznam, ktorý bude reprezentovať prepojenie týchto dvoch entít. Tabuľka bude vyjadrovať m:n vzťah, v dôsledku toho, že študent môže byť pridelený k viacerým zadaniam a zadanie môže byť pridelené viacerým študentom.

submitted_assignments – pri odovzdávaní prác študentov, je potrebné si záznamy o týchto prácach uchovávať, k čomu bude slúžiť nasledovná tabuľka. Okrem vlastného identifikátora, ktorý bude zároveň primárnym kľúčom, sa tu bude nachádzať aj identifikátor zadania, ku ktorému je práca odovzdávaná a identifikátor študenta, ktorý prácu odovzdal. Nakoľko práca je súbor, bude potrebné si pri ňom uchovávať cestu

a názov. Pre validitu práce bude nevyhnutné obsiahnuť aj informáciu o IP adrese a dátume odovzdania.

results – tabuľka bude uchovávať všetky hodnotenia odovzdaných prác. Záznam v tabuľke bude obsahovať bodové hodnotenie, komentár, identifikátor odovzdaného zadania, identifikátor vyučujúceho, ktorý bude prácu hodnotiť a identifikátor hodnotenia ako primárny kľúč.

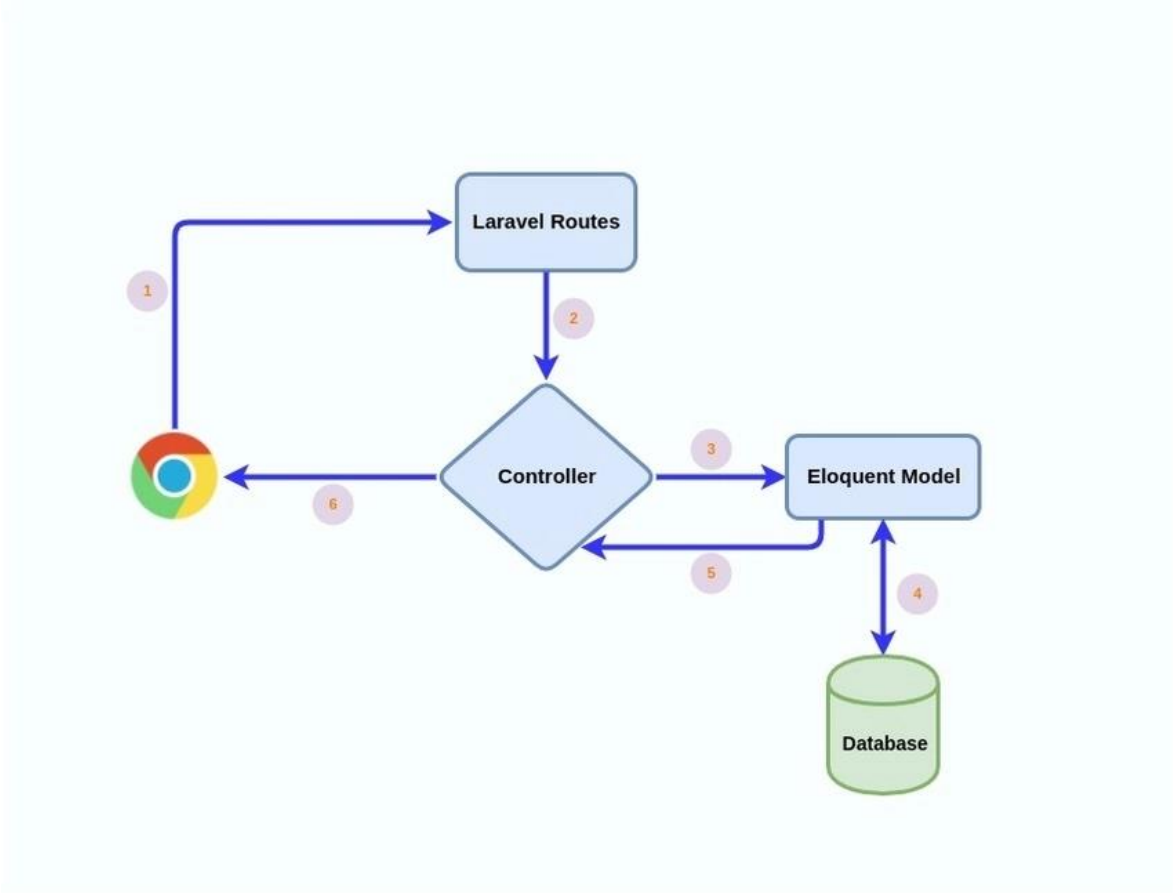
compared_pairs – po vykonaní kontroly originality prác, budú v prípade výraznej zhody dvojice prác uchovávané dáta v nasledujúcej tabuľke, ktorá bude definovaná sekvenčne generovaným primárnym kľúčom, percentom zhody a identifikátormi pre obe z dvojice porovnávaných prác.



Obrázok 8: Aktuálny databázový model

3.4 Architektúra systému

Hlavným komponentom systému bude webový server, ktorý bude od používateľov prijímať HTTP požiadavky a zhromažďovať dáta. Na strane klienta sa následne vytvorí HTML dokument na základe dát, prijatých zo servera. Pre uchovávanie dát bude využívaný PostgreSQL databázový systém. Prostredníctvom verejnej služby internet bude možná interakcia medzi užívateľom a aplikáciou cez zvolený prehliadač. Užívateľské rozhranie (front-end aplikácie) bude implementované pomocou open-source JavaScript knižnice React podporujúcej jednostránkové single-page aplikácie, CSS pravidiel a HTML značiek s využitím JSX. Serverové rozhranie (back-end aplikácie) bude implementované s využitím open-source frameworku Laravel v jazyku PHP, ktorý pracuje na architektúre MVC a na prácu s databázou využíva nástroj Eloquent ORM. V našej práci budeme však využívať len model a controller a vrstvu view, teda vrstvu pohľadu, nám bude vytvárať knižnica React, ako sprostredkovateľ bude využívaný architektonický štýl pre sieťovú komunikáciu REST API.



Obrázok 9: REST API - LARAVEL architektúra [2]

4 Použité technológie

V nasledovnej časti práce si priblížime technológie, ktoré boli využívané pri vývoji a samotnej tvorbe systému. Zahrnutý bude hlavný účel, krátky popis a konkrétne použitie v našej aplikácií.

4.1 DbDiagram

DbDiagram je bezplatný softvér na kreslenie databázových diagramov, ktorý využíva populárny DBML (Database Markup Language). [3] Nástroj sme využili pri tvorbe databázového modelu, kvôli intuitívnosti a prehľadnosti pri práci s ním.

4.2 PostgreSQL

PostgreSQL je open-source objektovo-relačný systém, ktorý využíva a rozširuje jazyk SQL v kombinácií s mnohými funkciami, ktoré podporujú vykonávanie rôznych dátových úloh. [4] Tento voľne dostupný systém využívame pre uchovávanie všetkých aktuálnych aj budúcich dát.

4.3 Laravel

Laravel je open-source PHP framework, pre tvorbu webových aplikácií. V súčasnosti sa jedná o najrozšírenejší PHP framework, vďaka robustnému súboru knižníc, ktorým disponuje. [5] Pre tento framework sme sa rozhodli aj na základe veľkej komunity vývojárov a programátorov, ktorí ho používajú. Medzi ďalšie výhody môžeme zaradiť nenáročnosť pochopenia fungovania. Zakladá si na priehľadnom kóde a opiera sa o rozsiahlu dokumentáciu na svojich oficiálnych stránkach. Framework využívame pri celkovej výstavbe back-end časti aplikácie. Je navrhnutý a vystavaný na princípe MVC (Model-View-Controller) architektúry.

4.3.1 MVC Architektúra

Model je vyjadrením entity dátového modelu, prostredníctvom modelu je zabezpečená interakcia s databázovou tabuľkou s využitím Eloquent ORM, kde každá tabuľka korešponduje s konkrétnym modelom.

Pohľad je výstupom, ktorý obsahuje šablóny s HTML kódom. Túto časť architektúry Laravelu v našej práci však využívať nebudeme.

Controller je časť s ktorou komunikuje užívateľ, model aj pohľad. Zabezpečuje prepojenie jednotlivých komponentov systému a je zvykom, že pre každú entitu existuje vlastný ovládač zahŕňajúci funkcie spojené s danou entitou. V našej práci bude slúžiť rovnako na komunikáciu medzi back-end a front-end, sprostredkovaný webovou službou REST API a HTTP operáciami GET, POST, PUT a DELETE. [6]

4.3.2 Routing

Medzi ďalšie dôležité súčasti tohto frameworku môžeme zaradiť smerovanie. HTTP požiadavku od užívateľa ako prvý spracuje router, ktorý prejde zoznamom definovaných ciest a pokiaľ nájde zhodu, na základe parametrov zistí, na aký ovládač a akú metódu smeruje. Ešte predtým ako zavolá príslušný ovládač a predá mu dáta môže prejsť požiadavka cez middleware triedu. Tá môže slúžiť napríklad na autentifikáciu užívateľa, či má právomoci na vykonanie odoslanej požiadavky. [5]

4.3.3 Composer

Ďalší nástroj, ktorý spolupracuje s Laravelom je Composer. Pomocou neho môže užívateľ vytvoriť projekt a inštalovať ďalšie knižnice. Rovnako slúži na správu týchto knižníc a balíčkov, o ktoré si užívateľ dokáže jednoducho požiadať pomocou príkazového riadku cez kľúčové slová: ”composer require [názov knižnice]”. Do zdrojového priečinku composer.json sú následne zaznamenávané všetky závislosti bez potreby dodatočnej konfigurácie a linkovania knižníc. [7]

4.3.4 Artisan

Rozhraním príkazového riadku v Laraveli je Artisan. Obsahuje sadu príkazov, ktoré pomáhajú pri tvorbe jednotlivých komponentov a samotnej webovej aplikácie. [7]

4.4 JWT

JSON webový token je určený na zabezpečenie koncových bodov API. Tento otvorený štandard umožňuje dvom stranám bezpečne posielať údaje a informácie ako objekty JSON. Tieto informácie možno overiť a dôverovať im, pretože sú digitálne podpísané. Na základe vygenerovaného tokenu je možné autorizovať a overovať klientov, ktorí majú prístup k prostriedkom API. [8] Tento spôsob autentifikácie pomocou tokenu je nazývaný aj bearer authentification, ktorý máme zahrnutý aj v našom systéme. Funguje na

princípe vygenerovaného reťazca – tokenu, ktorý generuje server ako odpoveď na žiadosť prihlásenia. Klient tento token pri svojich požiadavkách musí odosielať v autorizačnej hlavičke. [9]

4.5 Bootstrap

Bootsrap je bezplatný open-source fornt-end framework pre vytváranie webových aplikácií. Je založený na HTML, CSS a JavaScripte, na uľahčenie vývoja responzívnych webov. Responzívny dizajn zabezpečí prispôsobenie zobrazenia stránky na základe veľkosti a orientácie obrazovky užívateľa. [10]

4.6 React

React je JavaScript knižnica od spoločnosti Facebook určená na tvorbu užívateľských rozhraní. Z hľadiska MVC architektúry zastupuje práve jednu vrstvu, a to vrstvu pohľadu, v ktorej ponúka užívateľovi výstup s požadovanými dátami.

Základným stavebným prvok knižnice sú komponenty, ktoré predstavujú rôzne znovu použiteľné HTML elementy so zapuzdrenou funkcionalitou. Skladaním takýchto dielov vzniká komplexná aplikácia s užívateľským rozhraním. Komponenty majú svoje vlastnosti (props) a svoj vlastný stav (state). Tento deklaratívny spôsob práce s dátami aplikácie vedie k predvídateľnejšiemu chovaniu a jednoduchšiemu ladeniu.

V našej práci využívame framework pre jeho účely využitia pri tvorbe single-page aplikácií. Ďalšou výhodou, ktorú ponúka je JSX (JavaScript + XML), čo umožňuje písať HTML priamo do JavaScriptu s drobnými odlišnosťami oproti klasickému HTML. [11]

4.6.1 NPM

Rovnako aj pri Reacte využívame správcu balíčkov, ktorým je v tomto prípade Node Package Manager. Dokáže inteligentne spravovať závislosti medzi jednotlivými modulmi. Užívateľ si dokáže balíčky stiahnuť cez príkazový riadok a kľúčové slová: „npm install [názov balíčka]“. [12]

4.7 JPlag

JPlag je systém, ktorý sa snaží hľadať podobnosti medzi viacerými súbormi zdrojových kódov. Týmto spôsobom dokáže zistiť softvérový plagiát. Neporovnáva len bajty textu, ale dokáže rozoznať aj zhodu v syntaxi a štruktúre programu, vďaka čomu je odolný voči mnohým pokusom o maskovanie podobností medzi plagiátmi. Aktuálna softvérová podpora je pre programovacie jazyky Java, C#, C, C++. Je navrhnutý na detekciu podobnosti najmä medzi študentskými riešeniami, kvôli čomu ho využívame aj v našom systéme. [13]

5 Implementácia

V časti implementácia sa budeme zaoberať už samotnými postupmi pri tvorbe nášho systému s využitím spomínaných technológií. Rovnako prejdeme postupne cez hlavné celky našej kompozície. Do tejto časti budú navyše zahrnuté aj obrázky častí kódu a grafického prostredia aplikácie.

5.1 Databáza

Na konštrukciu dátového modelu sme použili nástroj dbdiagram slúžiaci na tvorbu diagramov, vzťahov medzi entitami a písanie krátkeho kódu. Na začiatku sme si stanovili dve hlavné tabuľky, tabuľku používateľov a tabuľku pre zadania, od ktorých sa budeme odvíjať. Zjednotili sme si všetky potrebné informácie, ktoré chceme zaznamenávať do databázy. Zhromaždené informácie sme triedili pre jednotlivé tabuľky a vytvárali z nich atribúty. Pri tvorbe sme sa držali základných princípov vytvárania databázy, všetky duplicitné údaje sme sa snažili eliminovať a každý záznam v tabuľke je unikátny na základe kľúča. Vzťahy medzi jednotlivými entitami sme vytvárali na základe relačných princípov a potrieb nášho systému. Po záverečnej optimalizácií bol vygenerovaný skript na export do PostgreSQL databázy. Skript sme nahrali na server cereyn.fri.uniza.sk, na ktorom nám aktuálne funguje celá databáza.

5.2 Úvod do projektu

Ako vývojové prostredie pre back-end aj front-end časť aplikácie sme používali Visual Studio Code, ktorý poskytuje možnosť jednoduchého stiahnutia ďalších rozšírení, ktoré nám v ďalšej práci uľahčovali písanie kódu pomocou klávesových skratiek a grafického členenia kódu.

Náš projekt sa skladá z dvoch aplikácií. Prvá pre back-end, kde pracujeme s frameworkom Laravel a zároveň sa pripájame k PostrgeSQL databáze. Druhá pre front-end, kde využívame výhody frameworku React. Hlavnou triedou tejto časti je trieda App.js, ktorá bude zabezpečovať prekresľovanie komponentov a zastrešovať fungovanie našej jedno-stránkovej klientskej aplikácie.

5.3 Autentifikácia

Autentifikácia v našom systéme je riešená pomocou JSON webového tokenu, inšpirovaná postupom na webovej stránke [8].

5.3.1 Server

Inštalácia samotného balíčka bola vykonaná pomocou Composera a viacerých príkazov. V prvom rade bolo potrebné vytvoriť model užívateľa - User, ktorý bude prepojený s tabuľkou užívateľov – users v databáze, z ktorej budú čerpané údaje pre autentifikáciu. Následne ako bolo spomínané, každý model by mal mať aj svoj ovládač, v našom prípade teda UserController. Ovládaču boli pridané tri funkcie:

login(Request $request) – preberá ako parameter požiadavku od klienta, v tomto prípade to bude požiadavka na prihlásenie, ktorá bude obsahovať dáta – prihlasovacie meno a heslo. Dáta najskôr prejdú validáciou, či zadal užívateľ oba údaje. V prípade, že nebol nejaký z údajov vyplnený, vráti funkcia chybovú hlášku v konkrétnom znení. Ak boli tieto podmienky splnené pokúsi sa trieda JWTAuth vytvoriť token, v prípade zhody údajov s údajmi v nejakom zázname tabuľky používateľov. V prípade neúspechu vráti príslušnú chybovú hlášku. Ak bol užívateľ autentifikovaný na základe údajov, do záznamu v databáze sa mu priradí vygenerovaný token. Rovnaký token je zároveň odosielaný ako odpoveď na stranu klienta spolu so statusom o úspešnom priebehu požiadavky.

getAuthenticatedUser() – funkcia, ktorá má na starosti vrátiť autentifikovaného užívateľa na základe analýzy tokenu.

logout(Request $request) – ako parameter preberá požiadavku od klienta na odhlásenie spolu s dátami o tokene užívateľa. Trieda JWTAuth status tokenu zmení na neplatný a v odpovedi klientovi odošle správu o úspešnosti akcie.

Na ochranu API ciest využívame služby JWT Middleware, ktorý zachytáva chyby tokenov, na základe ktorých sú odosielané príslušné chybové hlášky používateľom. Do priečinka routes, sme následne pridali cesty pre jednotlivé funkcie, aby bolo z HTTP požiadavky zreteľné, aký ovládač a aká funkcia sa má zavolať. Tie cesty, ktoré nepotrebujeme, aby podliehali autentifikačnej ochrane sa budú nachádzať mimo middleware zabezpečenia. K ostatným cestám budú môcť pristúpiť len prihlásení užívatelia.

5.3.2 Klient

Na front-end časti bolo potrebné vytvoriť komponent, ktorý bude zahŕňať formulár s položkami pre prihlasovacie meno a heslo spolu s tlačidlom, kde po kliknutí naň sa tieto údaje budú posielať na server. Vytvorili sme teda komponent Login, ktorý tieto požiadavky zastrešuje. Užívateľské rozhranie bolo vytvárané, ako bolo spomínané v predošlej kapitole prostredníctvom JSX a logické časti ďalej už len v klasickom JavaScripte. Okrem prihlasovacieho komponentu sme pre účely autentifikácie, ale aj pre účely budúcich požiadaviek vytvorili tri pomocné triedy – helpers.

ErrorHelper – trieda obsahuje funkciu getAllErrors(errors), ktorá ako parameter preberá pole chýb, odoslaných na stranu klienta, ktoré následne mapuje a vracia ako položky zoznamu.

AuthHelper – trieda obsahuje množstvo funkcií, pre prácu s autentifikovaným užívateľom. Vytvára a vracia inštanciu užívateľa, dokáže získať užívateľa na základe vygenerovaného tokenu, a teda vrátiť všetky potrebné informácie o aktuálne prihlásenom užívateľovi. Rovnako spravuje lokálne úložisko, kde ukladá, vyberá a odoberá užívateľský token.

ApiHelper – trieda obsahuje asynchrónnu funkciu getApiResponse(koncový bod, metóda, dáta, typ obsahu) s týmito parametrami, ktorá zastrešuje HTTP požiadavku na server spolu hodnotami parametrov, pre príslušné údaje.

Údajmi sú:

URL – http://localhost:8000/api/ + koncový bod uvedený ako parameter.

Metóda – GET, POST, PUT alebo DELETE, rovnako z parametra funkcie.

Dáta – dáta odosielané na server, uvedené ako parameter.

Typ obsahu – application/json, pokiaľ parametrom nie je určené inak.

Hlavička – bude obsahovať typ autentifikácie + token prihláseného užívateľa.

```
import axios from 'axios'
import AuthHelper from './AuthHelper'

export async function getApiResponse(endpoint, method, data = null, contentType = 'application/json') {
  return await axios.request({
    method: method,
    url: 'http://localhost:8000/api/' + endpoint,
    data: data,
    headers: AuthHelper.getInstance().getCurrentUser() != undefined ||  AuthHelper.getInstance().getCurrentUser() != null
    ? AuthHelper.getInstance().getAuthHeaders() : null,
    contentType: contentType,
  })
}
```

Obrázok 10: Trieda ApiHelper

5.3.3 Proces prihlásenia a odhlásenia

Priebeh procesu prihlásenia spočíva v tom, že užívateľ vyplní formulár, kde zadá svoje prihlasovacie údaje, následne zvolí možnosť prihlásenia, čím sa zavolá spomínaná funkcia getApiResponse(), ktorá odošle na server HTTP požiadavku na konkrétny koncový bod, metódou post s dátami formulára. Na serveri požiadavku prevezme konkrétny ovládač (UserController), ktorý zavolá funkciu login(), ktorá na základe prijatých dát autentifikuje užívateľa a vygeneruje token. Token následne zapíše do príslušného atribútu autentifikovanému užívateľovi a odosiela na stranu klienta ako odpoveď. Klient vytvorí inštanciu prihláseného užívateľa, do lokálneho úložiska si uloží token z odpovede a presmeruje užívateľa na úvodnú stránku s informáciami o predmete. Ešte pred samotným presmerovaním sú na stranu klienta načítané všetky potrebné dáta z databázy prostredníctvom GET požiadavky na server s príslušným koncovým bodom. V prípade neúspešnosti prihlásenia, odošle server klientovi odpoveď s konkrétnou chybovou hláškou, ktorá je užívateľovi zobrazená na danej stránke, kde prihlasovanie prebieha.

V prípade odhlásenia užívateľa, je na server odosielaná požiadavka príslušnému ovládaču na funkciu logout(). Danému užívateľovi je zo záznamu v databáze odobratý token. Server následne odosiela odpoveď so správou o úspešnosti odhlásenia. Na strane klienta sa odoberie z úložiska token a aktuálne prihlásený užívateľ sa nastaví na hodnotu null. Užívateľ bude v tom momente presmerovaný na prihlasovaciu stránku.

```php
public function login(Request $request) {
        $validator = Validator::make($request->all(), [
            'password' => 'required',
            'ldap_login' => 'required',
        ], [
            'password.required' => 'Zadajte heslo',
            'ldap_login.required' => 'Zadajte vaše univerzitné LDAP meno',
        ]);

        if($validator->fails()){
                return response()->json($validator->errors(), 400);
        }

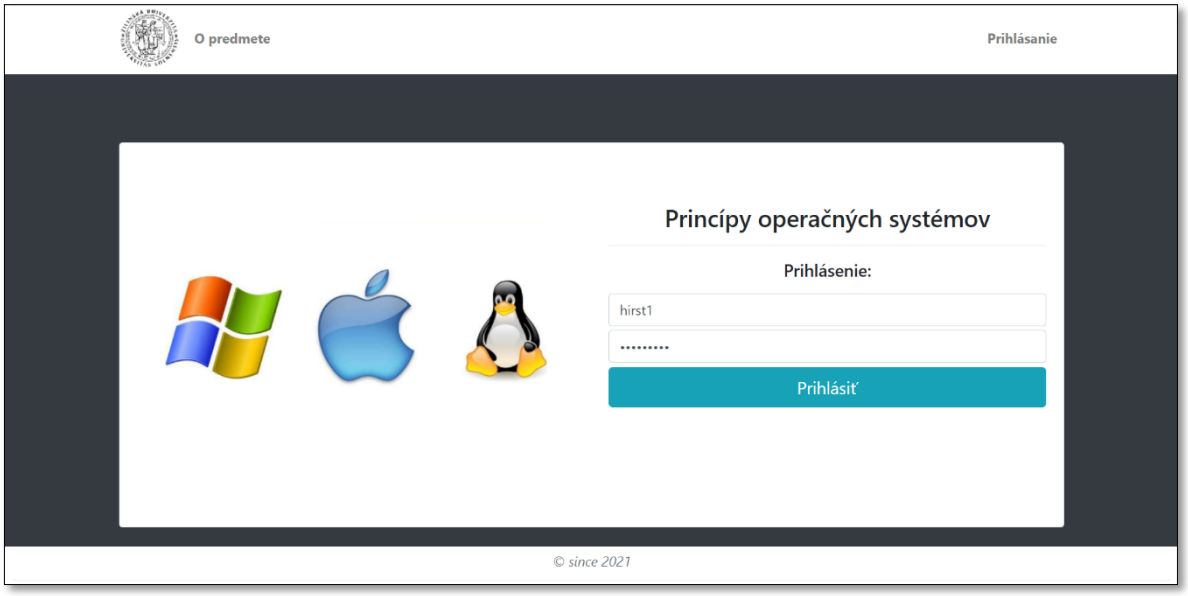
        $token = JWTAuth::attempt($request->only('ldap_login', 'password'));

        try {
            if ($token == null) {
                return response()->json(['errors' => ['Neplatné prihlasovacie údaje']], 400);
            }
        } catch (JWTException $e) {
            return response()->json(['errors' => ['Nie je možne vytvoriť token']], 500);
        }

        $user = User::where('ldap_login', $request->all()['ldap_login'])->first();
        $user->api_token = $token;
        $user->save();

        return response()->json(['token' => $token], 200);
}
```

Obrázok 11: UserController - funkcia login()



Obrázok 12: Prihlasovanie

5.4 Učiteľské rozhranie

Po úspešnej autentifikácií sú užívateľovi na základe role poskytnuté príslušné právomoci. Pri tvorbe rozhraní sme postupovali podľa požiadaviek a vytvorených scenárov. Ako prvým sme sa zaoberali rozhraním pre vyučujúceho, ktoré pozostávalo z hlavných funkcionalít ako tvorba a modifikácia zadaní, prideľovanie študentov a hodnotenie odovzdaných prác.

5.4.1 Zadania

Každý vyučujúci má právomoci pre tvorbu nového zadania. Na jeho vytvorenie je užívateľovi poskytnutý komponent s formulárom a tlačidlom pre uloženie. Pri všetkých formulároch je na strane klienta a servera zabezpečená validácia dát. Dáta sú následne odosielané metódou POST na server, kde požiadavku spracováva konkrétna funkcia TaskControllera – store(). Tu dochádza k overeniu, či požiadavku vytvoril vyučujúci a na základe prijatých dát k zápisu nového záznamu do databázy, a teda samotnému vytvoreniu. V prípade úspechu je v odpovedi zo servera odosielané nové zadanie, ktoré sa na klientskej strane lokálne pridá k zoznamu všetkých zadaní a nie je teda potrebné opätovné načítavanie dát z databázy. V opačnom prípade klient dostane odpoveď s konkrétnou chybovou hláškou.

```
async addNewTask(task) {

  const addedTask = await getApiResponse('tasks/store', 'post', task)

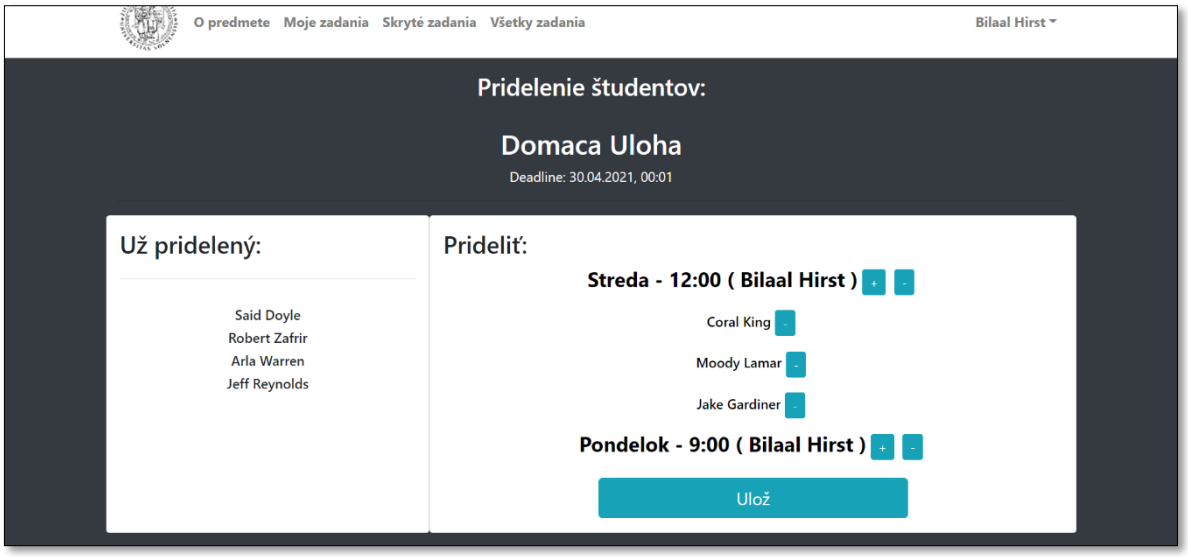
  this.setState(state => {
    return {
      allTasks: [...state.allTasks, addedTask.data],
      status: 1,
    }
  })
}
```

Obrázok 13: Požiadavka na server o vytvorenie nového zadania

Okrem vytvárania aplikácia umožňuje aj ďalšie modifikácie, akými sú úprava zadaní, skrývanie a mazanie. V prípade editácie je funkčnosť veľmi podobná vytváraniu. Užívateľ zmení údaje vo formulári, ktorý je už vyplnený momentálnymi dátami a uloží zmeny, ktoré sú ďalej odosielané a spracované na serveri príslušným ovládačom. V prípade, že k zadaniu nie sú priradení žiadny študenti je zadanie možné zmazať.

Vo všetkých týchto prípadoch sa užívateľovi zobrazí kontextové okno s informáciou o vykonanej akcii. Vyučujúci má rovnako možnosť zadanie skryť, ak nechce, aby úlohy daného zadania hodnotili ostatní vyučujúci alebo ak zadanie ďalej nie je aktuálne a nechce ho mať zobrazené medzi novými zadaniami.

Veľmi dôležitou funkciou, ktorá nasleduje je prideľovanie zadaní študentom. Užívateľovi je zobrazený zoznam jeho cvičení, po jeho otvorení sa zobrazia všetci študenti daného cvičenia, ktorý sú automaticky pridaní do poľa potenciálnych študentov na priradenie. Zobrazených študentov je možné jednotlivo alebo skupinovo odoberať. Uložením svojho rozhodnutia, odošle užívateľ pole všetkých zobrazených študentov spolu s identifikátorom daného zadania na server. Príslušný ovládač zavolá funkciu assignTask(), ktorá cyklusom for prejde celým poľom študentov prijatých z požiadavky, ktorých jednotlivo priraďuje k zadaniu, ktoré je získané z databázy na základe identifikátora zahrnutého v požiadavke. Na stranu klienta je odoslané práve dané zadanie spolu s referenciou na aktualizovanú tabuľku pridelených zadaní študentom. Klient je po úspešnom vykonaní požiadavky presmerovaný na stránku s tabuľkou pridelených študentov k danému zadaniu. Ak bol omylom priradený študent, ktorého vyučujúci nepožaduje pre konkrétne zadanie, dokáže ho jednoducho odobrať. Túto akciu je možné vykonať len dovtedy, kým študent prácu pre dané zadanie neodovzdá.



Obrázok 14: Prideľovanie zadania študentom na strane klienta

```php
public function assignTask(Request $request) {

    $user = JWTAuth::parseToken()->authenticate();

    if ($user->role != 't')
    {
        return response()->json(['status' => 'unauthorized'], 400);
    }

    $task = $request->get('task_id');
    $array = array();

    for ($i = 0; $i < count($request->chosenStudents); $i++) {
        $temp = $request->chosenStudents[$i];
        $user = User::where('user_id', Arr::get($temp, 'user_id'))->first();
        $user->stud_tasks()->attach($task);

        $array[$i] = User::with('schedules')
        ->with('stud_tasks')
        ->with('enrolled_student')
        ->with('submitted_assignments.result')
        ->with('submitted_assignments.compared_pair')
        ->where('user_id', Arr::get($temp, 'user_id'))
        ->first();
    }

    $task = Task::with('stud_tasks')
    ->with('submitted_assignments')
    ->where('task_id', $request->get('task_id'))
    ->first();

    return response()->json(['task' => $task, 'users' => $array], 201);
}
```

Obrázok 15: Spracovanie požiadavky na pridelenie zadania študentom na serveri

5.4.2 Kontrola originality prác a hodnotenie

Medzi ďalšiu funkciu, ktorú bude potrebné mať pre vyučujúcich k dispozícii je hodnotenie. Užívateľovi bude zobrazená tabuľka študentov priradených k danému zadaniu spolu s časom a stavom hodnotenia.

Pri danom zadaní bude možné jedenkrát všetky odovzdané práce od študentov podrobiť kontrole originality. Táto funkcionalita je však umožnená len pri zadaniach semestrálnych prác a programátorských písomiek. Na stránke sa bude nachádzať tlačidlo, ktoré danú kontrolu spustí. Kliknutím naň sa zavolá funkcia getApiResponse(), s konkrétnym koncovým bodom, zahŕňajúcim identifikátor zadania, ktorá metódou PUT

odošle požiadavku na server. Požiadavku preberie ovládač TeacherController, ktorý ju odošle svojej funkcií – checkDuplicates(). V rámci tejto funkcie sa vykoná systémový príkaz, ktorý nad priečinkom obsahujúcim všetky odovzdané práce daného zadania zavolá nástroj JPlag, ktorý porovná tieto práce, vyhodnotí pre všetky dvojice percento zhody a zapíše výsledky do csv súboru. Následne s využitím balíčka Maathwebsite je možné tento súbor čítať a riadky súboru sú odosielané ako parameter funkcií model(). Nakoľko jeden riadok prislúcha jednej práci a jej dvojiciam, kde sú páry zoradené od najvyššej zhody zostupne, zoberie sa z každého riadku prvá dvojica s najväčšou hodnotou zhody. Pre každú prvú dvojicu je takto v tabuľke compared_pairs v databáze vytvorení nový záznam. Hodnota, či už pre práce daného zadania bola vykonaná detekcia zhody sa nastaví na hodnotu pravda. V odpovedi servera klientovi je odosielaný status 200, ktorý vyjadruje, že požiadavka bola úspešne vykonaná. Na klientskej strane sa užívateľovi v prípade výskytu následne zobrazia dvojice študentov s najvyššími hodnotami zhody.

Ďalej bude práce možné hodnotiť. Užívateľovi je zobrazený komponent s názvom zadania, menom vybratého študenta, tlačidlom pre stiahnutie študentovej práce a formulárom s položkami pre dosiahnutý počet bodov a komentár. Po vyplnení sú jeho dáta odosielané na server, spracované funkciou príslušného ovládača. Tu sa vytvorí nový záznam do databázovej tabuľky results, aktualizuje sa dosiahnutý počet bodov študenta v tabuľke enrolled_students a v odpovedi je odosielaný práve študent, ktorý bol ohodnotený. Dáta tohto študenta sú následne na strane klienta lokálne aktualizované. V tabuľke študentov sa pri danom študentovi zmení stav hodnotenia na „hodnotené“.

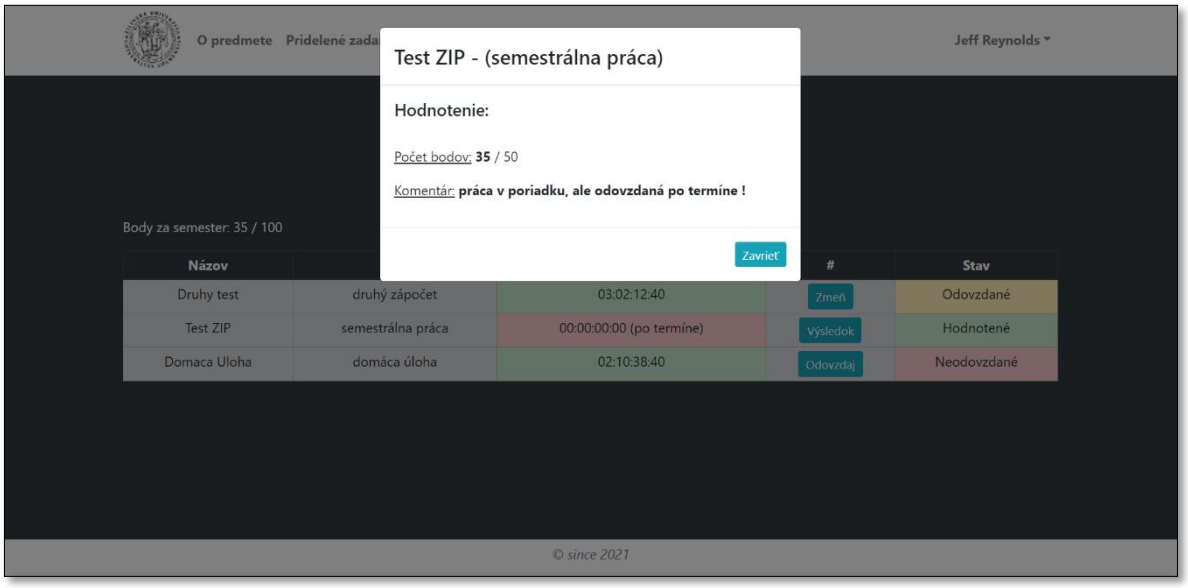
5.5 Študentské rozhranie

Študent, podobne ako vyučujúci bude mať zabezpečenú jedinečnú funkcionalitu, ktorá bude zahŕňať možnosť odovzdávania prác a náhľadu svojich výsledkov. Na stránke pridelených zadaní bude zobrazená tabuľka všetkých pridelených zadaní, ktoré má študent vypracovať spolu s údajmi o type práce, zostávajúcom čase na odovzdanie a stave odovzdávania. Okrem tabuľky bude študentovi zobrazený aj aktuálny bodový zisk zo semestra.

5.5.1 Odovzdávanie prác a výsledky

Pre odovzdávanie prác bude užívateľovi s rolou študent zobrazený komponent s formulárom pre údaje o IP adrese, z ktorej je práca nahrávaná a miesto pre priloženie súboru. Oba údaje sú vyžadované, a teda v prípade uloženia zmien bez vyplnenia niektorého z údajov, príde zo servera odpoveď s chybovou hláškou o nevyplnenom údaji. V prípade vyplnenia oboch položiek, budú dáta odoslané na server, prejdú validáciou a vytvorí sa nový záznam v tabuľke submitted_assignments. Na základe typu zadania bude nahratý súbor uložený v príslušnom priečinku na serveri. Ako odpoveď je klientovi odosielaný študent odkazujúci na aktualizovanú tabuľku svojich odovzdaných zadaní. Dáta študenta sú po prijatí aktualizované na strane klienta a stav odovzdávania bude zmenený na „odovzdané“. Ak študent odovzdal prácu po vypršaní limitu na odovzdanie, bude položka deadline podfarbená červenou, v opačnom prípade zelenou farbou.

Pokiaľ bude práca ohodnotená vyučujúcim, zobrazí sa pri nej študentovi tlačidlo „výsledok“. Po kliknutí sa zjaví kontextové okno s bodovým hodnotením a komentárom k práci študenta.



Obrázok 16: Náhľad hodnotenia práce študenta

Záver

Hlavným cieľom našej práce bolo vytvoriť aplikáciu, ktorá umožní učiteľom predmetu Princípy operačných systémov spravovať zadania pre študentov, hodnotiť práce a kontrolovať ich originalitu. Funkcionalita pre študentov bude pozostávať z nahrávania svojich vypracovaných zadaní a zobrazenia svojich priebežných hodnotení. Okrem hlavných požiadaviek boli v práci implementované aj doplnkové funkcie. Celý systém je navrhnutý tak, aby práca s ním bola pre užívateľa intuitívna a nenáročná.

Práca spĺňa všetky vopred stanovené požiadavky vyplývajúce z cieľov zadania bakalárskej práce. Pri samotnej realizácií sme aplikovali poznatky v oblasti tvorby webových aplikácií ako aj v oblasti práce s frameworkami. Na zabezpečenie sme použili typ autentifikácie bearer, pomocou JWT. Komunikáciu klienta so serverom je sprostredkovaná pomocou služby REST API. Pre kontrolu originality prác sme využívali voľne dostupný nástroj JPlag. Vo vývojovom prostredí Visual Studio Code sme súčasne pracovali na dvoch projektoch, pre back-end a front-end. Na strane servera sme využívali framework Laravel, postavený na MVC architektúre. Na klientskej strane sme využívali React freamework určený pre tvorbu užívateľských rozhraní, ktorý zabezpečuje reaktivitu medzi vstupmi užívateľa a aplikáciou. Oba frameworky nám v značnej miere ušetrili čas a zabezpečili pohodlnosť pri práci.

Veríme, že náš systém bude z hľadiska funkcionality a prehľadnosti pre potreby fakultného predmetu uspokojujúci. V budúcnosti plánujeme našu prácu zjednotiť s aplikáciou slúžiacou na prihlasovanie študentov na cvičenia. Aby bolo všetko na jednom mieste, chceli by sme zahrnúť rovnako aj možnosť prihlasovania na skúšku. Ďalšou víziou je rozšírenie aplikácie aj pre iné predmety s podobnou štruktúrou testovania študentov.

Zoznam použitej literatúry

[1] „Moodle,“ [Online]. Available: https://moodle.org/?lang=sk. [Cit. 7 4 2021].

[2] „RESTful-API-using-Laravel:MVC-Architecture,“ [Online]. Available: https://cms-assets.tutsplus.com/uploads/users/1795/posts/29442/image/RESTful-API-using-Laravel:MVC-Architecture.jpg. [Cit. 11 4 2021].

[3] „dbdiagram.io - Docs,“ [Online]. Available: https://www.notion.so/dbdiagram-io-Docs-Public-d405d7d9efd4419d9b6402eb6feb8cd1. [Cit. 12 4 2021].

[4] „PostgreSQL - About,“ [Online]. Available: https://www.postgresql.org/about/. [Cit. 12 4 2021].

[5] „ITnetwork : Lekce 1 - Úvod do Laravel frameworku pro PHP,“ [Online]. Available: https://www.itnetwork.cz/php/laravel/uvod-do-laravel-frameworku-pro-php. [Cit. 12 4 2021].

[6] „ITNetwork - MVC architektura,“ [Online]. Available: https://www.itnetwork.cz/navrh/mvc-architektura-navrhovy-vzor. [Cit. 12 4 2021].

[7] „TutorialsPoint: Laravel - Overview,“ [Online]. Available: https://www.tutorialspoint.com/laravel/laravel_overview.htm. [Cit. 12 4 2021].

[8] „Pusher: Laravel and JWT,“ [Online]. Available: https://blog.pusher.com/laravel-jwt/. [Cit. 12 4 2021].

[9] „Swagger: Bearer Authentication,“ [Online]. Available: https://swagger.io/docs/specification/authentication/bearer-authentication/. [Cit. 12 4 2021].

[10] „WhatIs - Bootstrap,“ [Online]. Available: https://whatis.techtarget.com/definition/bootstrap. [Cit. 13 4 2021].

[11] „ITnetwork: Uvod do react,“ [Online]. Available: https://www.itnetwork.cz/javascript/react/zaklady/uvod-do-react/. [Cit. 13 4 2021].

[12] „NPMJS: NPM,“ [Online]. Available: https://docs.npmjs.com/cli/v7/commands/npm. [Cit. 13 4 2021].

[13] „KIT - JPlag,“ [Online]. Available: http://jplag.ipd.kit.edu/. [Cit. 14 4 2021].

Zoznam príloh

Príloha A Používateľská príručka

Príloha B DVD

Prílohy

Príloha A: Používateľská príručka

Obsahuje manuál k používaniu aplikácie.

Príloha B: DVD

Priložené DVD obsahuje:

- Prácu v elektronickej podobe (formát PDF)
- Zdrojové kódy aplikácie
- Používateľskú príručku
- Odkaz na repozitár